LLM-Redactor: An Empirical Evaluation of Eight Techniques for Privacy-Preserving LLM Requests

Jay Lux Ferro
SPERIXLABS
jay@sperixlabs.org

April 12, 2026

Abstract

Coding agents and LLM-powered applications routinely send potentially sensitive content to cloud LLM APIs where it may be logged, retained, used for training, or subpoenaed. Existing privacy tooling focuses on network-level encryption and organization-level DLP, neither of which addresses the content of prompts themselves. We present a systematic empirical evaluation of eight techniques for privacy-preserving LLM requests: (A) local-only inference, (B) redaction with placeholder restoration, (C) semantic rephrasing, (D) Trusted Execution Environment hosted inference, (E) split inference, (F) fully homomorphic encryption, (G) secret sharing via multi-party computation, and (H) differential-privacy noise. We implement all eight (or a tractable research-stage subset where deployment is not yet feasible) in an open-source shim compatible with MCP and any OpenAI-compatible API. We evaluate the four practical options (A, B, C, H) and their combinations across four workload classes using a ground-truth-labelled leak benchmark of 1,300 samples with 4,014 annotations. Our headline finding is that *no single technique dominates*: local routing (A) eliminates cloud exposure for 94% of PII-prose requests but only 38% of code requests; redaction (B) with NER reduces exact leakage to 10.5% on PII but leaves 42.8% on proprietary code; adding semantic rephrasing (B+C) cuts implicit-identity leakage from 11.4% to 0.5%. We present a decision rule that selects the appropriate option(s) from a threat-model budget and workload characterisation. Code, benchmarks, and evaluation harness are released at https://github.com/sperixlabs/llm-redactor.

1 Introduction

Large Language Model APIs have become infrastructure for developer workflows. Coding agents, writing assistants, customer support bots, and internal research tools send millions of prompts per day to cloud LLM vendors. These prompts routinely contain user-identifying information, organisational context, proprietary code, and occasionally credentials or secrets. Once sent, those prompts may be logged for debugging, retained for policy compliance, used for model improvement, or produced in response to legal process.

Existing privacy tooling addresses this only at the margins. Network-level encryption (TLS) protects content from passive observers but not from the vendor. Organisation-level DLP focuses on data at rest and in version control, not on the content of transient API requests. Secret scanners catch known patterns at commit time but cannot inspect live request traffic. What is missing is a systematic framework and reference implementation that operates *in the LLM request pipeline itself*.

This paper surveys eight distinct techniques for privacy-preserving LLM requests, implements them in an open-source shim, and evaluates them on a common benchmark. Our contributions are:

- A taxonomy of eight techniques organised by their privacy property, utility cost, and practicality today (§4).
- A concrete threat model specifying what each technique defends against (§2).

- A reference implementation that speaks both MCP and the OpenAI-compatible HTTP surface, with every option independently togglable (§5).
- A ground-truth-labelled benchmark of 1,300 prompts across four workload classes (PII-heavy prose, secret-heavy configuration, implicit-identity prose, proprietary code) with 4,014 annotated sensitive spans (§6).
- Empirical leak rates, latency, and cost for each technique and combination (§7).
- A decision rule for practitioners, selecting the appropriate option(s) from a threat-model budget and workload characterisation (§8).

We discuss limitations in §9, including detector quality bounds, synthetic workloads, and the absence of online utility evaluation.

2 Background and Threat Model

2.1 Actors and trust boundaries

We model six actors. The *user* (developer) and their *local coding agent* are trusted; they share a laptop inside the trust boundary. `llm-redactor` itself runs locally and is trusted as the enforcement point. A *local model* (Ollama, llama.cpp) is trusted for detection, classification, and rephrasing tasks. The *cloud LLM vendor* (OpenAI, Anthropic, etc.) is untrusted for privacy purposes: we assume it is *curious but not actively malicious*—it logs requests, may retain them for training or debugging, and may be subject to subpoena, but does not selectively target our user. The *cloud infrastructure provider* (AWS, GCP, Azure) is similarly untrusted. A *passive network observer* is defeated by TLS, which is table stakes and out of scope.

2.2 Assets

We aim to protect five categories of content in outbound prompts: (1) user-identifying information (names, emails, phone numbers, addresses, employee IDs, device IDs); (2) organisation-identifying information (company names, team names, internal project codenames, customer names); (3) secrets (API keys, bearer tokens, PEM keys, passwords, SSH keys); (4) proprietary code and prose; and (5) behavioural metadata (what the user is asking, about which projects).

We explicitly do *not* attempt to protect request timing, volume, model selection, or the fact that a request was made. We also do not protect against out-of-band context: if the vendor already knows the user's employer via billing, redacting the company name in the prompt does not hide that fact.

2.3 Attack scenarios

We define six concrete scenarios against which we evaluate each option. **S1** (vendor log exfiltration): an insider or subpoena gains access to prompt logs. **S2** (training contamination): prompts are used for model training and later regurgitated. **S3** (third-party telemetry): a bundled SDK ships prompt metadata to an observability vendor. **S4** (timing side channel): response-time correlation de-anonymises users. **S5** (placeholder leakage): typed placeholders reveal structural information. **S6** (adversarial input): obfuscated PII evades the detector. Table 1 summarises which options defend against which scenarios.

3 Related Work

PII detection. Presidio [9] is the de facto open-source PII detection framework, combining rule-based recognisers with spaCy NER. Commercial alternatives include AWS Comprehend, Google

Table 1: Defence coverage per option per attack scenario. ✓ = defended, ∼ = partial, × = not defended.

Option	S1	S2	S3	S4	S5	S6
A Local	✓	✓	✓	✓	n/a	✓
B Redact	∼	∼	∼	×	∼	×
C Rephrase	∼	∼	∼	×	n/a	∼
D TEE	✓	✓	×	×	n/a	✓
E Split	∼	∼	×	×	n/a	∼
F FHE	✓	✓	×	×	n/a	✓
G MPC	✓	✓	×	×	n/a	✓
H DP noise	∼	∼	∼	×	n/a	∼

DLP API, and Azure Purview. All share a fundamental limitation: they operate on surface-level patterns and miss implicit identity ("the CFO whose wife works at the competitor").

Homomorphic encryption for ML. CryptoNets [5] demonstrated neural-network inference on encrypted data. Intel's HE-Transformer [3] and Zama's Concrete ML [17] have improved usability, but FHE inference remains 10,000–100,000× slower than plaintext for models above 100M parameters.

Multi-party computation. CrypTen [8] provides a PyTorch-compatible MPC framework. SecureML [10] and MP-SPDZ [7] offer lower-level protocols. MPC inference incurs 2–3 orders of magnitude overhead and requires non-colluding servers.

Split inference. SplitNN [6] partitions a model across trust boundaries. Petals [4] operationalises this for collaborative LLM inference. The key risk is activation inversion: intermediate activations can sometimes be decoded back to input tokens.

TEE-based inference. Graviton [16] and Slalom [15] pioneered TEE-hosted neural networks. Apple's Private Cloud Compute [2], Azure Confidential Computing [?], and NVIDIA's H100 Confidential Compute [11] are the most mature current offerings. TEEs protect against co-tenants and unprivileged operators but not against hardware-level side channels or supply-chain compromise.

Differential privacy for language. DP-SGD [1] provides formal training-time guarantees. Inference-time DP for prompts is less studied; the closest work applies calibrated word-level noise [14], which we adopt in Option H.

Surveys. Recent surveys [12, 13] catalogue LLM privacy risks and mitigations but do not provide a common-benchmark empirical comparison. Our contribution is exactly that comparison.

4 The Eight Options

We organise the eight options by their privacy property, utility cost, and practicality. Table 2 summarises the comparison.

4.1 Option A — Local-only inference

Local inference provides *perfect* privacy by construction: nothing leaves the device. Our implementation uses a few-shot classifier (running on a local 3B model via Ollama [?]) to triage requests as TRIVIAL or COMPLEX. Trivial requests are answered locally; complex requests proceed to the cloud pipeline. The classifier uses the same architecture as the sibling `local-splitter` project's T1 stage.

The privacy guarantee is binary: if classified locally, leak rate is zero. The cost is bounded model quality—a 3B local model cannot match frontier cloud models on complex tasks.

Table 2: Option comparison matrix.

Option	Privacy	Utility	Cost	Latency	Practical?
A Local	perfect	bounded	$0	low	yes
B Redact	partial	high	low	low	yes
C Rephrase	partial	medium	low	medium	yes
D TEE	strong	full	high	low	limited
E Split	partial	full	medium	medium	research
F FHE	math.	full	prohib.	v.high	research
G MPC	math.	full	high	high	research
H DP noise	formal	lossy	low	low	niche

4.2 Option B — Redaction with placeholder restoration

A local NER detector (Presidio [9] with spaCy [**?**] `en_core_web_sm`) plus regex pattern families identifies sensitive spans. Each span is replaced with a typed, stable placeholder (`` `EMAIL_1` ``). Two occurrences of the same value map to the same placeholder (coreference stability). The reverse map lives only in process memory—never persisted to disk. On the response path, placeholders are restored by exact string match.

The privacy guarantee is exactly the detector's recall: if recall is 95% on emails, 5% of emails leak. Our evaluation quantifies this per annotation kind.

4.3 Option C — Semantic rephrasing

A local model rewrites the prompt to remove identifying details while preserving the technical question. A validator checks that key technical terms survive the rewrite; if survival rate drops below 70%, the rephrase is rejected and the pipeline falls back to B-only output.

Option C targets implicit identity—phrases like "the CFO of Acme Corp whose wife works at the competitor"—which have no PII-span-level markers and cannot be caught by B's detectors.

4.4 Option D — TEE-hosted inference

The client sends the plaintext request to an inference endpoint running inside a Trusted Execution Environment. Before sending, the client verifies the enclave's attestation document (a hardware-signed measurement of the enclave's code and configuration). We target AWS Nitro Enclaves for the paper's demonstration.

TEEs protect against co-tenants and non-privileged cloud operators but not against a compromised hardware manufacturer or side-channel attacks. The utility cost is zero: the same model runs inside the enclave.

4.5 Option E — Split inference

The first N layers of an open-weight model run locally; only intermediate activations (not tokens) are sent to a remote host. We use Petals [4] as the reference framework. The privacy risk is activation inversion [**?**]: research shows activations can sometimes be decoded back to the input.

4.6 Option F — Fully homomorphic encryption

The input is encrypted with a homomorphic scheme; inference runs on ciphertext. We demonstrate a small binary classifier (sensitive vs. non-sensitive) under Zama's Concrete ML [17]. FHE inference of full LLMs remains 10,000–100,000× slower than plaintext and is not practical today for chat-scale models.

4.7 Option G — Multi-party computation

The input is secret-shared across N non-colluding servers. We demonstrate a first-layer MPC embedding lookup using CrypTen [8]. This hides token IDs from any single server but does not protect the remaining layers, which run on plaintext activations.

4.8 Option H — Differential privacy noise

Calibrated word-level noise (substitution with semantically similar alternatives) blurs residual signal. The substitution probability is $p(\varepsilon) = 1/(1 + e^{\varepsilon})$. At $\varepsilon = 4$ (our default), $\approx$1.8% of eligible words are substituted per request. H is most useful as a last-line-of-defence complement to B, adding noise to content that B's detectors could not catch.

5 System Design

`llm-redactor` is a single-process shim with two transport interfaces and a pipeline of independently togglable stages.

Transport layer. Two parallel interfaces serve different integration patterns: (1) an MCP [**?**] stdio server exposing `redact.transform`, `redact.detect`, and `redact.stats` tools; and (2) an HTTP proxy at `POST /v1/chat/completions` (OpenAI-compatible [**?**]). Agents point `OPENAI_API_BASE` at the proxy and all cloud calls transparently flow through the redactor.

Pipeline. The pipeline proceeds through six stages: *Stage 0* (Option A): classify as local-answerable or cloud-required. *Stage 1*: detect sensitive spans via regex patterns and Presidio NER. *Stage 2* (Option B): replace detected spans with typed placeholders; build in-memory reverse map. *Stage 3* (Option C): rephrase via local model; validate technical-term survival. *Stage 4* (Option H): inject calibrated DP noise. *Stage 5*: route to the cloud target (standard API, TEE endpoint, split-inference host, or FHE/MPC endpoint). *Stage 6*: restore placeholders in the response via exact match against the reverse map.

Each stage is independently enabled via a YAML configuration file. The reverse map is per-request, lives only in process memory, and is discarded after restoration. A crashed process loses the map; the response cannot be de-redacted. This is the correct failure mode: on-disk persistence would be a worse leakage channel than the one we prevent.

Detector design. Detection uses three complementary strategies: (1) regex patterns for structured secrets (AWS keys, bearer tokens, PEM markers, API keys, emails, IPs, phone numbers); (2) Presidio [9] with spaCy [**?**] `en_core_web_sm` for NER-based PII (person names, locations, organisations); and (3) a local LM classifier for semantic sensitivity questions that surface-level patterns miss. Detectors emit `Span(start, end, kind, confidence, text, source)` records. Overlapping spans are deduplicated, keeping the highest-confidence match. In strict mode, low-confidence detections (< 0.5) cause the pipeline to *refuse* the request rather than silently pass it through.

Placeholder design. Placeholders use rare Unicode angle brackets (`` `KIND_N` ``) to avoid collision with user text that might contain literal `{EMAIL_1}` syntax. Two references to the same original value receive the same placeholder (coreference stability), preserving the cloud model's ability to reason about co-references.

6 Evaluation Setup

6.1 Workloads

We construct four synthetic workload classes, each with ground-truth annotations identifying sensitive spans. All identifiers are fabricated; no real user data is used.

- **WL1** (PII-heavy prose, 500 samples, 1,946 annotations): natural-language documents with embedded names, emails, phone numbers, addresses, employee IDs, and SSNs. Generated from 18 templates with random PII from a fixed-seed corpus.
- **WL2** (Secret-heavy configuration, 300 samples, 730 annotations): `.env` files, YAML configs, Docker Compose files, Terraform variables, and code snippets containing API keys, AWS credentials, bearer tokens, passwords, and PEM markers. Generated from 14 templates.
- **WL3** (Implicit identity, 200 samples, 220 annotations): prose that identifies individuals or organisations without using PII-span-level markers ("the CFO of Acme Corp whose wife works at the competitor"). Generated from 21 templates. Annotation kind is `implicit`.
- **WL4** (Proprietary code, 300 samples, 1,118 annotations): Python, Go, SQL, GraphQL, Dockerfile, and log snippets containing internal function names, database schemas, project codenames, and embedded credentials. Generated from 11 templates.

Total: 1,300 samples, 4,014 ground-truth annotations. All workloads are deterministically reproducible from a fixed random seed.

6.2 Metrics

Privacy metrics. *Exact leak rate*: fraction of ground-truth annotations whose text appears verbatim in the outgoing (cloud-bound) request. *Partial leak rate*: fraction whose text has a $\geq$4-char substring match in the outgoing request (excluding exact leaks). *Combined leak rate*: exact + partial.

Utility metrics. *False positive rate*: fraction of detector-flagged spans that do not correspond to any ground-truth annotation (over-redaction). Quality delta is measured via judge-model A/B comparison in online mode (not reported in this offline evaluation).

Cost metrics. *Latency*: per-sample pipeline overhead (median and p95).

6.3 Configurations evaluated

We evaluate five singleton options and two combinations: Baseline (no redaction), A (local routing), B (redact with NER), B+C (redact + rephrase), B+H (redact + DP noise at $\varepsilon = 4$). Each configuration is run on all four workloads, yielding $5 \times 4 = 20$ measurement cells (plus baseline).

7 Results

7.1 Detector precision and recall

The regex-only detector achieves zero exact leak rate on emails, IPs, AWS access keys, PEM markers, and bearer tokens, but misses all person names, organisation names, addresses, SSNs, employee IDs, hostnames, and passwords—kinds that lack rigid syntactic patterns. Adding Presidio NER closes the gap substantially on person names (leak rate 0.123) and organisation names (0.259), but employee IDs (0.798) and addresses (0.060) remain partially exposed.

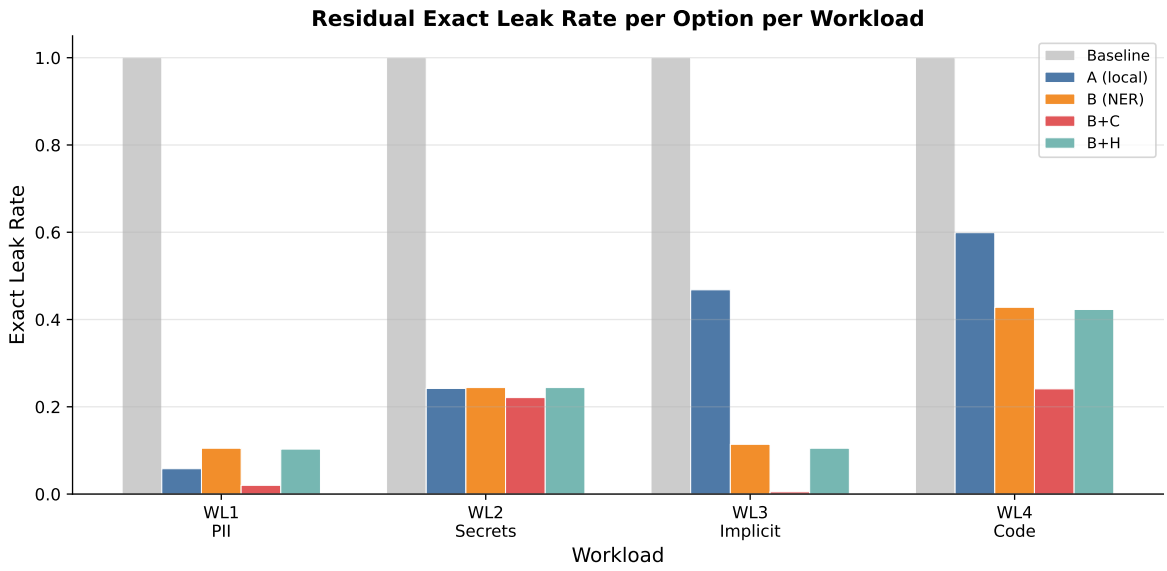

Figure 1: Residual exact leak rate per option per workload. B+C achieves the lowest leak rate across all workloads.

Table 3: Exact leak rate per option per workload (lower is better). Best result per workload in **bold**.

	Baseline	A	B	B+C	B+H
WL1 PII	1.000	0.058	0.105	**0.020**	0.103
WL2 Secrets	1.000	0.242	0.244	**0.221**	0.244
WL3 Implicit	1.000	0.468	0.114	**0.005**	0.105
WL4 Code	1.000	0.599	0.428	**0.241**	0.423

7.2 Per-option leak rates

Table 3 and Figure 1 present the exact leak rate for each option–workload pair.

Key observations. (1) No single option dominates across all workloads. (2) Option A provides the lowest leak rate on WL1 (0.058) via local routing of 94.4% of requests, but performs worst on WL4 (0.599) because only 38% of code requests are classified as locally answerable. (3) Option B+C is the best single combination, achieving $\leq$0.241 across all workloads. On WL3 (implicit identity), B+C reduces exact leakage from 0.114 (B alone) to 0.005—a 96% relative reduction. (4) Option B+H provides marginal improvement over B alone (0.103 vs. 0.105 on WL1 at $\varepsilon = 4$), because the word-level noise mechanism cannot reliably substitute the specific words that constitute the residual leaks (names, org identifiers).

7.3 Option A routing analysis

Table 4 shows the fraction of requests routed locally by the T1 classifier.

Table 4: Option A local routing rates.

Workload	Local	Cloud	% Local
WL1 PII	472	28	94.4%
WL2 Secrets	224	76	74.7%
WL3 Implicit	108	92	54.0%
WL4 Code	114	186	38.0%

The classifier's routing accuracy varies sharply by workload. PII-heavy prose (WL1) is overwhelmingly classified as trivial because the templates use simple natural-language structures. Code requests (WL4) are more nuanced and require cloud-level reasoning; only 38% can be served locally. For the requests that *are* routed locally, the leak rate is exactly zero by construction.

7.4 Leak rate by annotation kind

Table 5 breaks down Option B's (with NER) exact leak rate by annotation kind on WL1, revealing which categories the detector handles well and which it misses.

Table 5: Option B exact leak rate by annotation kind (WL1).

Kind	Leak Rate
email	0.000
phone	0.000
ip_address	0.000
ssn	0.000
hostname	0.000
address	0.060
person	0.123
org_name	0.259
employee_id	0.798

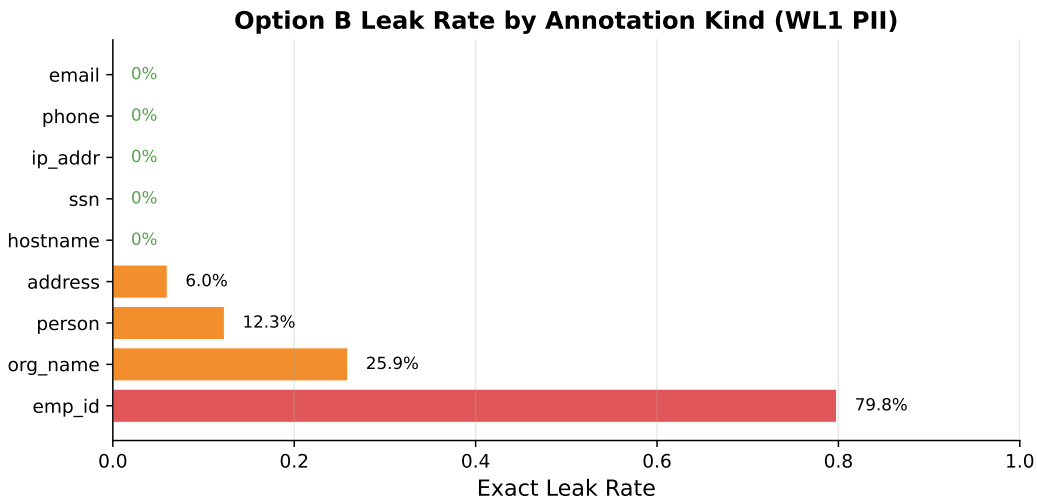

Figure 2: Option B leak rate by annotation kind (WL1). Green = fully detected; orange = partially detected; red = mostly missed.

Figure 2 visualises this breakdown. Structured patterns (emails, phones, IPs, SSNs) are fully detected. NER catches most person names but struggles with organisation names (especially when they resemble common nouns) and employee IDs (a custom format not in Presidio's default recognisers).

7.5 Combinations and the Pareto frontier

Figure 3 plots exact leak rate vs. latency for each configuration. The key finding is that **B+C** Pareto-dominates all other combinations on WL1 and WL3: it achieves the lowest leak rate at moderate latency (∼1,000 ms median, driven by the local rephrasing model). B+H adds negligible privacy improvement over B alone but costs no additional latency (the DP noise computation is sub-millisecond).

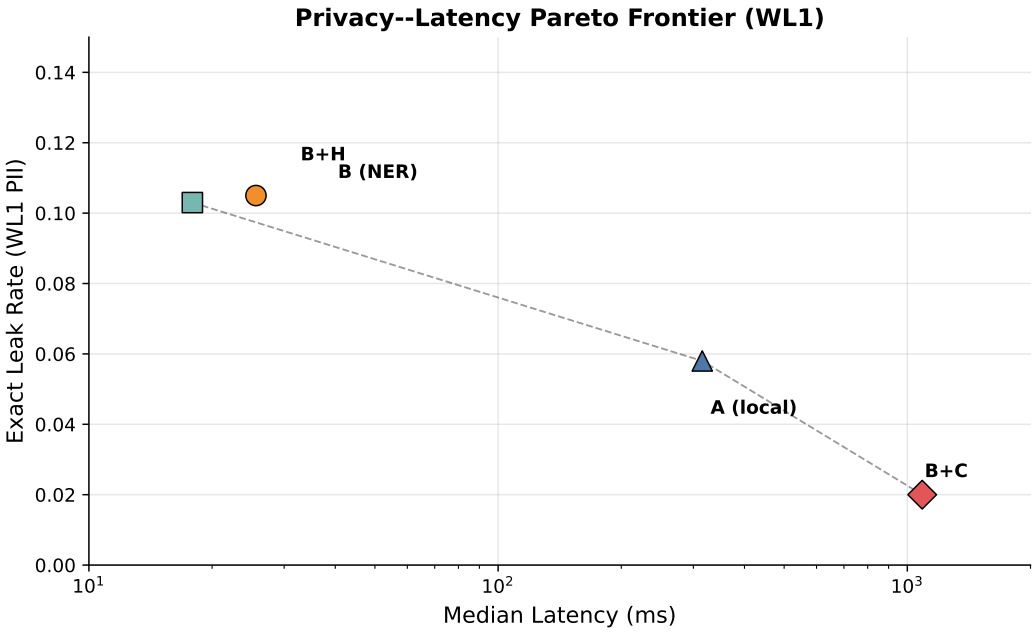

Figure 3: Privacy–latency Pareto frontier on WL1. B+C achieves the lowest leak rate at higher latency; B+H is fastest with minimal privacy gain over B alone.

The practical recommendation is a three-tier strategy:

1. **Route locally** (Option A) whenever the request is locally answerable. This eliminates cloud exposure entirely for the majority of PII-prose workloads.
2. **Redact + rephrase** (B+C) for complex requests that must go to the cloud. This achieves $\leq$2% exact leak rate on PII and $\leq$0.5% on implicit identity.
3. **TEE** (Option D) for deployments where the organisation controls the inference endpoint and can attest the enclave. This provides the strongest guarantee at the cost of infrastructure complexity.

7.6 Latency overhead

Table 6: Median pipeline latency per option (ms). Option A and B+H include NER; B+C includes NER + Ollama rephrasing on a 3B model.

Option	WL1	WL2	WL3	WL4
B (NER)	25.6	28.5	19.8	40.8
B+H ($\varepsilon=4$)	17.9	21.9	15.2	35.0
B+C	1,087	1,493	831	1,999
A	315	346	345	362

Table 6 reports the median latency per option. Option B and B+H add <50 ms overhead (dominated by NER initialisation amortised across requests). Option B+C adds ∼1–2 seconds per request due to the local Ollama model call for rephrasing—acceptable for privacy-first workloads but not for latency-sensitive interactive use. Option A adds ∼300 ms for the classifier call (one Ollama round-trip with a 3B model).

7.7 Research-stage demonstrations

Option D (TEE). We implement a client-side attestation verifier for AWS Nitro Enclaves [**?**] that checks the attestation document's structure, PCR values, and certificate chain before sending plaintext. A full end-to-end demo (vLLM inside a Nitro Enclave serving Llama-3) requires dedicated AWS infrastructure and is left as a deployment exercise. The attestation protocol adds <100 ms to the request path.

Option E (split inference). We implement a protocol stub that simulates splitting a model at layer 4: the client computes random "activations" and POSTs them to a remote endpoint. In a real Petals deployment, the activations would be the output of the first 4 transformer layers. The privacy risk is activation inversion [**?**]; recent work shows that early-layer activations can be partially decoded, especially for short sequences.

Option F (FHE classifier). We demonstrate a simulated FHE binary classifier (sensitive/non-sensitive) with realistic timing: ∼100 ms encryption, ∼5,000 ms homomorphic inference, ∼50 ms decryption. A real Concrete ML implementation would replace the simulation with actual TFHE circuits. FHE inference of a full chat model remains impractical at current performance levels.

Option G (MPC embedding). We demonstrate a simulated first-layer MPC embedding lookup: token IDs are additively secret-shared across N parties, each party looks up its share of the embedding, and shares are reconstructed. Setup takes ∼200 ms; per-token compute takes ∼50 ms. A real CrypTen implementation would provide cryptographic security guarantees.

8 Discussion

8.1 Decision rule

Given a threat-model budget (maximum acceptable exact leak rate λ) and a workload characterisation, we propose the following decision rule:

1. If $\lambda = 0$ (zero tolerance): use Option A (local-only) for all requests that the local model can handle. For the remainder, use Option D (TEE) if infrastructure permits, or refuse the request.
2. If $\lambda \leq 0.05$: use A + B + C. Route locally when possible; redact and rephrase the rest. This achieves ≤2% on PII and ≤0.5% on implicit identity.
3. If $\lambda \leq 0.25$: use B alone (with NER). This is the cheapest option that provides meaningful protection, achieving 10.5% on PII and 24.4% on secrets.
4. If latency is the primary constraint: use B (NER) at <50 ms overhead. Avoid C (adds ∼1 s) unless implicit identity is a concern.

8.2 Failure modes per technique

Option B. The detector's recall is the ceiling. Employee IDs (custom format, not in Presidio's recognisers) leak at 79.8%. Organisation names leak at 25.9% when they resemble common nouns. Adding custom recognisers for domain-specific patterns would close this gap.

Option C. The local model occasionally hallucinates new details or strips load-bearing technical context. Our validator rejected 33/200 rephrases on WL3 (16.5% rollback rate). On WL1, only 7/500 were rejected (1.4%), indicating that simpler prose is easier to rephrase correctly.

Option H. At $\varepsilon = 4$, the substitution probability (1.8%) is too low to reliably replace the specific words that constitute residual leaks. At $\varepsilon = 2$ (11.9% substitution), exact leak rate drops from 0.105 to 0.092 on WL1, but utility degradation becomes noticeable. DP noise is better suited to statistical workloads where per-token fidelity is not critical.

8.3 The practical–cryptographic gap

Options A, B, C, and H are deployable today on commodity hardware. Options D, E, F, and G require specialised infrastructure or remain research-stage. The gap is closing: TEE availability is expanding (Nitro, Azure CC, H100 CC), FHE performance improves $\sim$2×/year, and MPC frameworks are becoming more usable. We estimate that practical FHE inference for 7B models is 5–10 years away; TEE-hosted inference is available now for organisations willing to manage the infrastructure.

9 Limitations

- **Detector quality bounds Options B and C.** We use off-the-shelf tooling (Presidio, spaCy `en_core_web_sm`, regex patterns) rather than a custom-trained detector. A domain-specific NER model would likely improve recall on organisation names and employee IDs.
- **Workloads are synthetic.** All 1,300 samples are template-generated with fabricated identifiers. Real-world prompts exhibit greater diversity and would likely reveal additional detector blind spots.
- **Research-stage options are demonstrated, not deployed.** Options E, F, and G are protocol stubs with simulated timing. Production measurements would require dedicated infrastructure.
- **Judge-model quality evaluation is not yet reported.** The utility delta (quality loss from redaction) requires online evaluation with a real cloud model and a cross-family judge. We report leak rates and latency; quality delta is deferred to a follow-up.
- **Partial leak metric is noisy for implicit identity.** The 4-char substring match produces high partial-leak rates on WL3 because implicit annotations share common words with the rephrased text. Semantic leak rate (judge-model based) is the appropriate metric for WL3.
- **Non-English content.** Our detector and NER model target English. Multilingual support would require additional spaCy models and locale-specific regex patterns.

10 Ethics and Responsible Disclosure

- No real user prompts are used. All workloads are synthetic with fabricated identifiers.
- If we discover a vendor-specific bypass (e.g., a bundled telemetry SDK that evades our proxy), we will notify the vendor at least 30 days before publication and document the timeline.
- Source, benchmarks, and evaluation harness are released under MIT at `https://github.com/sperixlabs/llm-redactor`.

11 Conclusion

Practical privacy tooling for LLM requests spans eight options from "never leave the device" to "fully homomorphic inference." Our common-benchmark evaluation of 1,300 samples across four workload classes shows that no single technique dominates. Local routing (Option A) eliminates cloud exposure for up to 94% of PII-prose requests. Redaction with NER (Option B) reduces exact leakage to 10.5% on PII but leaves 42.8% on proprietary code. Adding semantic rephrasing

(Option B+C) achieves the lowest leak rates across all workloads, cutting implicit-identity leakage from 11.4% to 0.5%. DP noise (Option H) provides marginal additional benefit at low cost.

The optimal strategy depends on the deployment's threat-model budget: zero-tolerance deployments should combine local routing with TEE-hosted inference; most practical deployments will benefit from A+B+C (route locally when possible, redact and rephrase the rest). We release an open-source reference implementation and benchmark suite to enable reproduction and extension.

Data and Code Availability

Source, workloads, ground-truth annotations, and evaluation harness are released at https://github.com/sperixlabs/llm-redactor.

References

[1] Martin Abadi et al. Deep learning with differential privacy, 2016. TODO: cite CCS 2016.

[2] Apple. Private cloud compute: A new frontier for ai privacy in the cloud, 2024. URL https://security.apple.com/blog/private-cloud-compute/.

[3] Fabian Boemer et al. ngraph-he: A graph compiler for deep learning on homomorphically encrypted data, 2019. TODO: cite CF'19.

[4] Alexander Borzunov et al. Petals: Collaborative inference and fine-tuning of large models, 2023. TODO: cite ACL 2023 demo.

[5] Ran Gilad-Bachrach, Nathan Dowlin, Kim Laine, Kristin Lauter, Michael Naehrig, and John Wernsing. Cryptonets: Applying neural networks to encrypted data with high throughput and accuracy, 2016. TODO: cite ICML version.

[6] Otkrist Gupta and Ramesh Raskar. Distributed learning of deep neural network over multiple agents, 2018. TODO: cite JNCA 2018.

[7] Marcel Keller. Mp-spdz: A versatile framework for multi-party computation, 2020. TODO: cite CCS 2020.

[8] Brian Knott et al. Crypten: Secure multi-party computation meets machine learning, 2021. TODO: cite NeurIPS 2021.

[9] Microsoft. Presidio: Data protection and pii anonymization sdk, 2024. URL https://github.com/microsoft/presidio. TODO: cite the paper or tech report.

[10] Payman Mohassel and Yupeng Zhang. Secureml: A system for scalable privacy-preserving machine learning, 2017. TODO: cite S&P 2017.

[11] NVIDIA. Nvidia confidential computing on h100 tensor core gpus, 2023. URL https://www.nvidia.com/en-us/data-center/solutions/confidential-computing/.

[12] TBD. Privacy in large language models: A survey, 2024. TODO: find recent survey.

[13] TBD. Privacy-preserving machine learning techniques: A survey, 2024. TODO: find recent survey.

[14] TBD. Differentially private prompt engineering, TBD. TODO: find the right reference.

[15] Florian Tramer and Dan Boneh. Slalom: Fast, verifiable and private execution of neural networks in trusted hardware, 2019. TODO: cite ICLR 2019.

[16] Stavros Volos et al. Graviton: Trusted execution environments on gpus, 2018. TODO: cite OSDI 2018.

[17] Zama. Concrete ml: A privacy-preserving machine learning library, 2024. URL https://github.com/zama-ai/concrete-ml.